Module 3 | Introduction to TensorFlow & Keras

26 July 2025 20:05

What is TensorFlow ?

- ~~Library~~
- Framework
- ~~Ecosystem~~

What is a framework ?

A framework in software (including DL) is a comprehensive set of tools, libraries, rules and structures that provides everything you need to build, train and deploy models or applications

Tensorboard ??

Examples: TensorFlow, PyTorch, Django (web), Reactjs, Spring (Java)

" TensorFlow (TF) is a deeplearning framework."

TF is an open-source developed by Google Brain in 2015.

(- for their internal research and production)

It is widely used for building ML models | DL models due to its flexibility, scalability and ability to run on CPUs, GPUs and TPUs.

↳ (Tensor Processing Units by Google)

TF supports building models at multiple abstraction levels — from low level operations to high level APIs like tf.keras.

Tensorflow (low level)

Keras (High level API)

python Copy Edit

Tensorflow (low level)

python

```python
import tensorflow as tf
import numpy as np

# Dummy data
X = np.random.randn(100, 2).astype(np.float32)
y = (np.sum(X, axis=1) > 0).astype(np.float32).reshape(-1, 1)

# Initialize weights and biases
W1 = tf.Variable(tf.random.normal([2, 4]))
b1 = tf.Variable(tf.zeros([4]))
W2 = tf.Variable(tf.random.normal([4, 1]))
b2 = tf.Variable(tf.zeros([1]))

# Define model
def forward_pass(X):
    z1 = tf.matmul(X, W1) + b1
    a1 = tf.nn.relu(z1)
    z2 = tf.matmul(a1, W2) + b2
    output = tf.nn.sigmoid(z2)
    return output

# Loss and optimizer
def loss_fn(y_true, y_pred):
    return tf.reduce_mean(tf.keras.losses.binary_crossentropy(y_true, y_pred))

optimizer = tf.optimizers.Adam(learning_rate=0.01)

# Training loop
for epoch in range(100):
    with tf.GradientTape() as tape:
        y_pred = forward_pass(X)
        loss = loss_fn(y, y_pred)
    grads = tape.gradient(loss, [W1, b1, W2, b2])
    optimizer.apply_gradients(zip(grads, [W1, b1, W2, b2]))
    if epoch % 10 == 0:
        print(f"Epoch {epoch}, Loss: {loss.numpy():.4f}")
```

Keras (High level API)

python

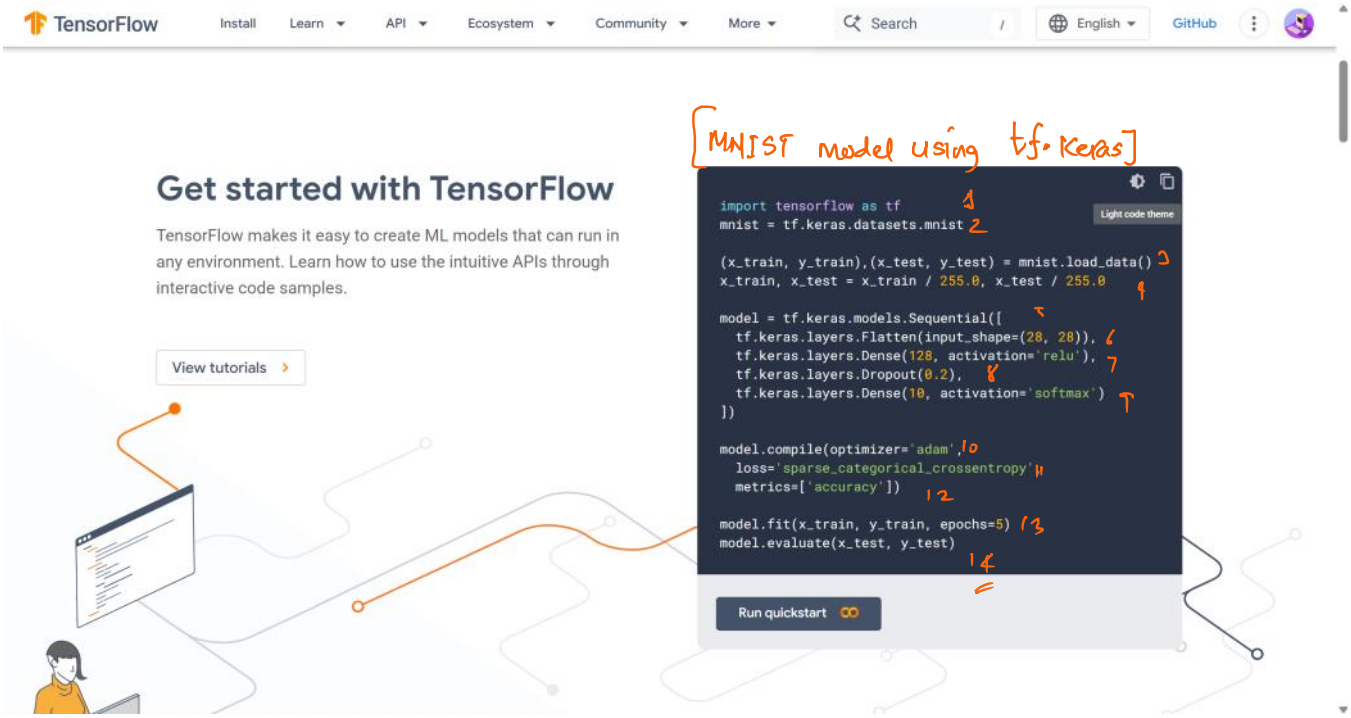
```python
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np

# Dummy data
X = np.random.randn(100, 2)
y = (np.sum(X, axis=1) > 0).astype(np.float32)

# Build model
model = keras.Sequential([
    layers.Dense(4, activation='relu', input_shape=(2,)),
    layers.Dense(1, activation='sigmoid')
])

# Compile
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# Train
model.fit(X, y, epochs=100, verbose=1)
```

MLP from scratch – APC. (even lower level)

INPUT LAYER ---> (1) HIDDEN LAYER ---> OUTPUT LAYER

```
class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size, learning_rate = 0.01, epochs=100):
        self.input_size = input_size # no. of input vars or features in the input layer
        self.hidden_size = hidden_size #no. of neurons in the hidden layer (in here we have only one hidden layer)
        self.output_size = output_size #no. of neurons in the output layer
        self.learning_rate = learning_rate #learning rate for the gradient descent step
        self.epochs = epochs #no. of training epochs

        ### INITIALIZE WEIGHTS AND BIASES

        ============================
        # INPUT LAYER TO HIDDEN LAYER
        ============================
        self.w1 = np.random.randn(self.input_size, self.hidden_size)*0.01 #random weights drawn from std. normal distribution
        self.b1 = np.zeros((1, self.hidden_size)) # zero bias values for the neurons in the hidden layer

        ============================
        # HIDDEN LAYER TO OUTPUT LAYER
        ============================
        self.w2 = np.random.randn(self.hidden_size, self.output_size)*0.01 #random weights drawn from std. normal distribution
        self.b2 = np.zeros((1, self.output_size)) # zero bias values for the neurons in the output layer

        ### LOSS & ACCURACY HISTORY for PLOTTING
        self.loss_history = [] #empty list initialized to store the losses during training epochs
        self.accuracy_history = [] #empty list initialized to store the accuracy values during training epochs


    ### ADD SOME ACTIVATION FUNCTIONS

    ###  ReLU activation function
    def relu(self, z):
        return np.maximum(0,z)

    ### Derivative of ReLU for backpropagation
    def relu_derivative(self, z):
        return np.where(z>0, 1, 0)

    ### Softmax activation function
    def softmax(self, z):
        exp_values = np.exp(z - np.max(z, axis=1, keepdims=True)) #subtract max for numerical stability
        return exp_values/np.sum(exp_values, axis=1, keepdims= True)


    ============================
    # 1.FORWARD PROPAGATION
    ============================
    def forward(self, X):

        ============================
        # INPUT LAYER TO HIDDEN LAYER
        ============================
        self.z1 = np.dot(X, self.w1) + self.b1 # compute z = wX + b1 B
        self.a1 = self.relu(self.z1) #plug z1 IT into ReLU activation function to get output as a1

        ============================
        # HIDDEN LAYER TO OUTPUT LAYER
        ============================
        self.z2 = np.dot(self.a1, self.w2) + self.b2 # compute z = w2a1 + b2 B
        self.probs = self.softmax(self.z2) #plug z2 IT into Softmax activation function to get output as a2
        return self.probs

    ============================
    # 2.COMPUTE LOSS & ACCURACY
    ============================
    ### Cross-entropy loss for multi-class classification
    def compute_loss(self, y_true, probs):
        loss = -np.mean(np.sum(y_true * np.log(probs), axis=1))
        return loss
    ### Compute accuracy
    def compute_accuracy(self, y_true, probs):
        predictions = np.argmax(probs, axis=1)
        true_labels = np.argmax(y_true, axis=1)
        return np.mean(predictions == true_labels)

    ============================
    # 3.BACKPROPAGATION
    ============================
    def backward(self, X, y):

        ## Using Batch Gradient Descent (BGD)
        ### Number of rows/training examples
        m = X.shape[0] #all rows in the data

        ### Gradients of the loss w.r.t. weights and biases of the output layer
        delta3 = self.probs - y #error at the output layer
        dW2 = np.dot(self.a1.T, delta3)/m #gradient of the loss w.r.t. weights of the output layer --> dW2
        db2 = np.sum(delta3, axis=0, keepdims=True)/m #gradient of the loss w.r.t bias of the output layer --> db2

        ### Gradients of the loss w.r.t weights and biases of the hidden layer
        delta2 = np.dot(delta3, self.w2.T) * self.relu_derivative(self.z1) #using derivative of ReLU
        dW1 = np.dot(X.T, delta2)/m #gradient of the loss w.r.t weights of the hidden layer --> dW1
        db1 = np.sum(delta2, axis=0, keepdims=True)/m #gradient of the loss w.r.t bias of the hidden layer --> db1

        ### Updating the parameters --> weights and biases across the layers (Hidden Layer & Output Layer)
        self.w2 -= self.learning_rate * dW2
        self.b2 -= self.learning_rate * db2
        self.w1 -= self.learning_rate * dW1
        self.b1 -= self.learning_rate * db1


    ============================
    # MLP TRAIN
    ============================
    def train(self, X,y):
        for epoch in range(self.epochs):

            ### 1. Forward pass
            probs = self.forward(X) #computes the prediction using forward pass method

            ### 2. Compute the loss and accuracy
            loss = self.compute_loss(y, probs)
            accuracy = self.compute_accuracy(y, probs)
            self.loss_history.append(loss)
            self.accuracy_history.append(accuracy)

            ### 3. Backward Pass
            self.backward(X,y)

            ### Print the progress after every 10 epochs
            if epoch % 10 == 0:
                print(f"Training Metrics: Epoch: {epoch}, Loss: {loss:.4f}, Accuracy: {accuracy:.4f}")
```

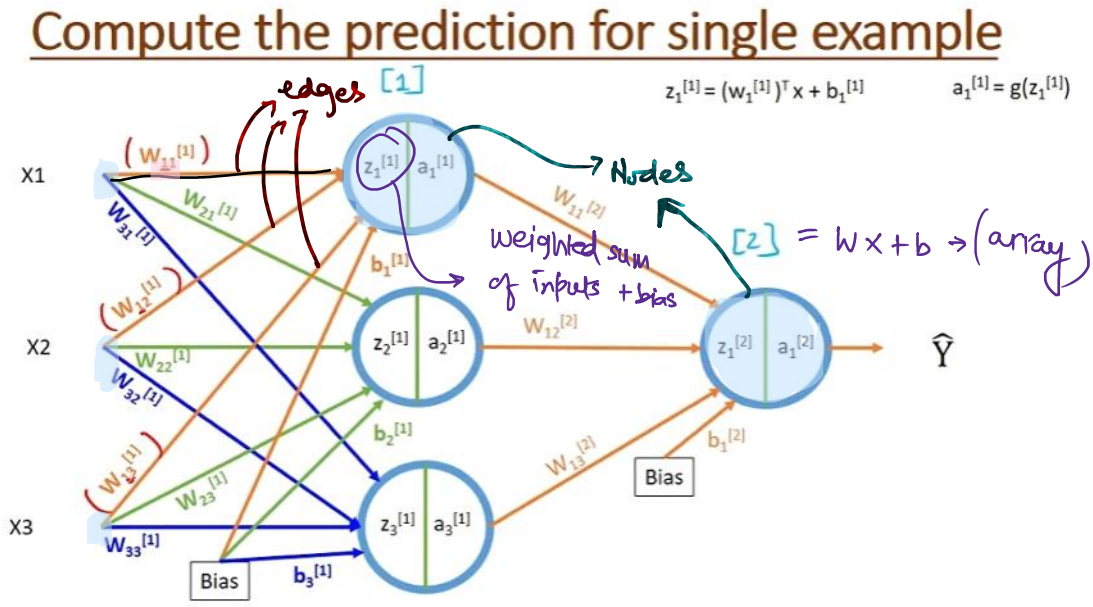
Below is the MLP architecture for nn1.net.run



https://www.tensorflow.org/

Tensor Flow

What do you mean by 'Tensor' and 'Flow' in TensorFlow?



In any n/w graph, there are nodes and edges

$$W = \begin{bmatrix} W_{11} & W_{12} & W_{13} & \cdots \\ W_{21} & W_{22} & W_{33} & \\ \cdots & & & \end{bmatrix}$$

↓

edges

edges

- edges in any n/w graph represent multi-dimensional arrays called TENSOR

Tensor → represents multi-dimensional data structure (n-dim arrays)

Flow → represents the way data moves through the n/w of operations

$$z_1 = W_1 x + b_1$$

$$f(z) \rightarrow a_1 \rightarrow a_1 W_2 + b_2$$

[Note: TensorFlow is **immutable** - tf.constant // **tf.Variable is mutable**]

↓ can be mutable or immutable depends on the type of Tensor.

Keras

What is Keras?
- → framework ✗
- → library / API → (High-level neural n/w API)
- → eco system ✗

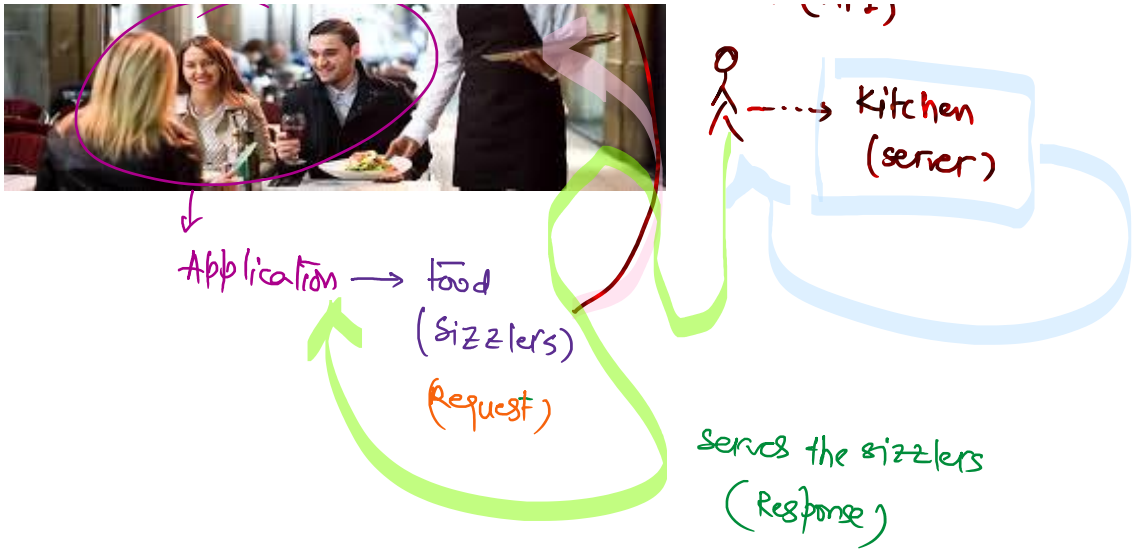
It is an open source high level neural n/w API (Application Programming Interface) developed by Francois Chollet and it released 2015.

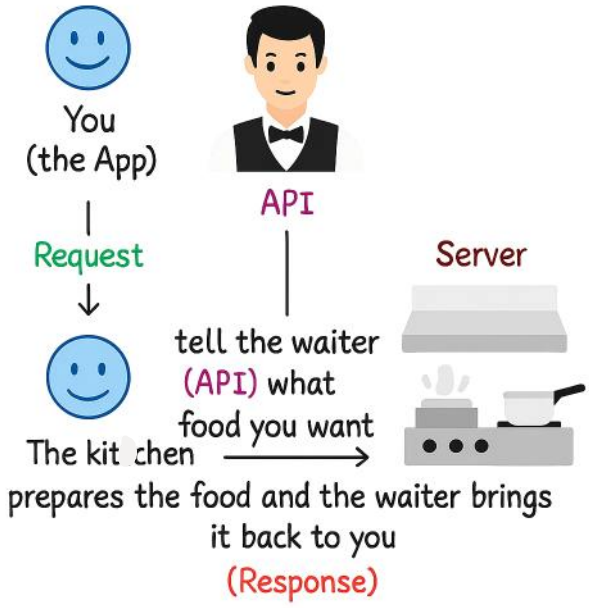
What's an API?

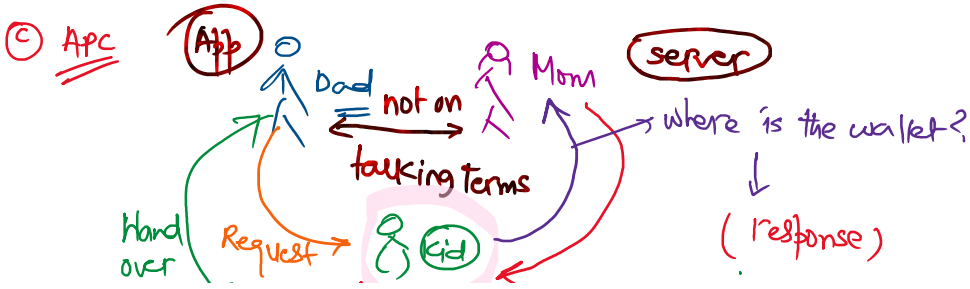
API : Application Programming Interface

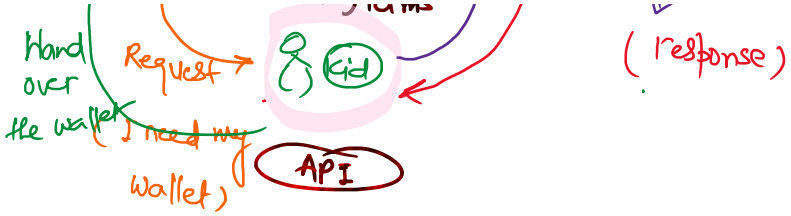
suppose you are going to a fine-dining restaurant on a fine saturday.

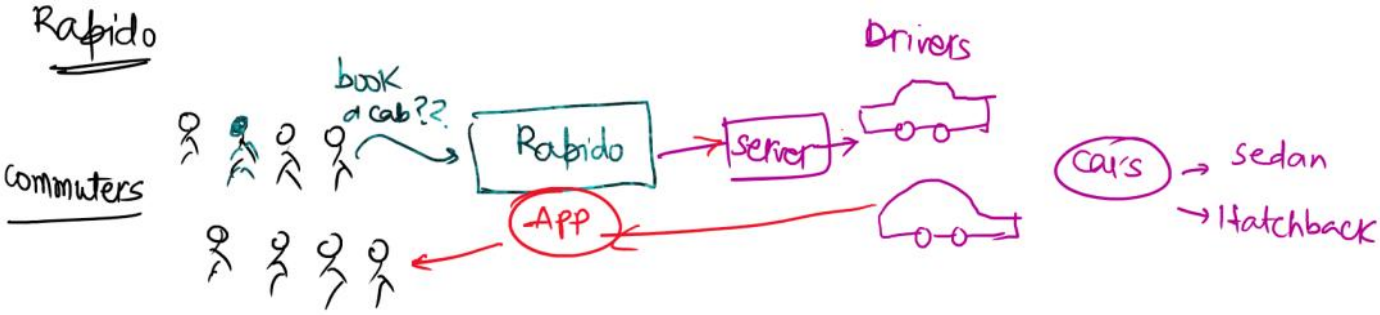
waiter (API)


Kitchen
(server)
Application → food
(sizzlers)
(Request)
serves the sizzlers
(Response)


You
(the App)
API
Request
Server
tell the waiter
(API) what
food you want
The kitchen
prepares the food and the waiter brings
it back to you
(Response)


© Apc
App
Dad
not on
talking terms
Mom
server
where is the wallet?
(response)
Hand over
Request
Kid





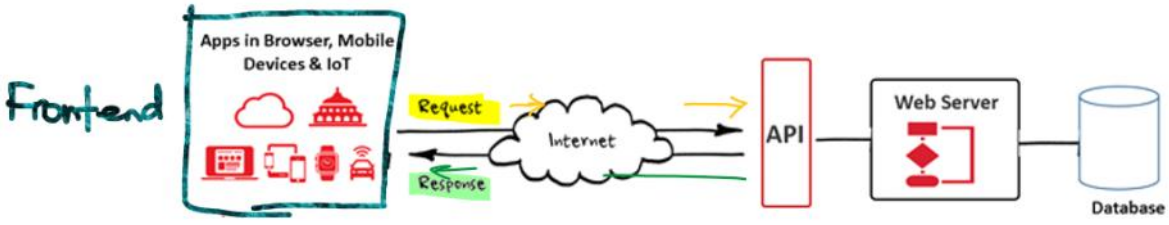
Run a command prompt:

Pick-up point - Full detailed address
Drop-point - Again full detailed address
Time -
Name -
PAX - 3
Contact
Email-ID
Car type

Do you your own search ..then talk to every guy --> finalize a cab --> trip is confirmed///

People will get 'CRAZY'

An API is a software which can be used by other softwares, to communicate with other softwares or even hardwares. It acts as a bridge between different softwares and devices.



API is a software intermediary that allows two applications or more than that to talk to each other.

Why APIs ??

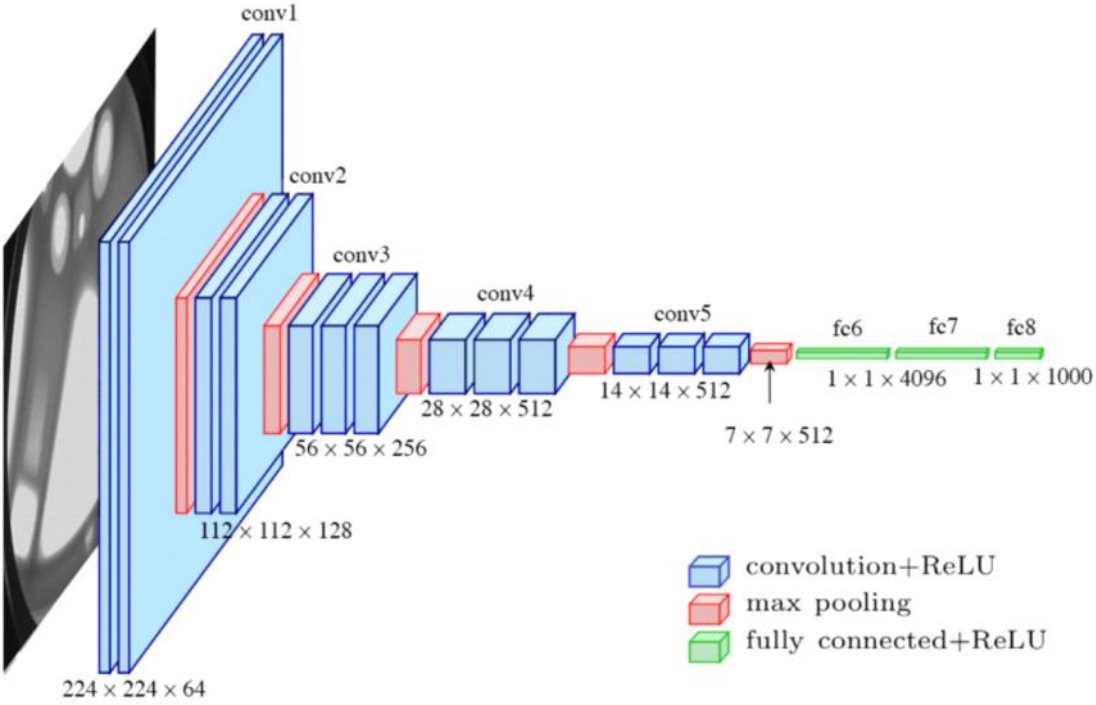
#① User-friendly – Easy to use and understand → making it ideal platform for the beginners.

#② Modularity – Building blocks for neural n/w models that's been saved after previously being trained on a large-scale datasets

Keras does provide several pre-trained models

- Resnet 50
- VGG 16
- YOLO v12
- YOLO v08
- Deepseek r1

VGG16 is a deep convolutional neural network (CNN)



(Pre-trained model)

TensorFlow vs Keras comparison

	TensorFlow	Keras
• Type:	DL framework	High level API | library for DL
• Level:	Low-level + High-level (via Keras)	High-level only
• origin:	Developed by Google Brain in 2015	Developed by Francois Chollet in 2015
• Purpose:	Build, train, deploy deep learning models	Quickly build, train and experimenting
• Flexibility:	very flexible, allows for custom operations and full control.	designed for simplicity, less control over low-low-level operations
• Standalone:	Yes (can be run without Keras)	Keras is tightly ~~integrated~~ integrated with TensorFlow